Análisis de la plataforma académica en microservicios

Proyecto final · Ingeniería de Software III

Diego López Camacho — Universidad Industrial de Santander

9 de noviembre de 2026

1. Cómo trabajé este análisis

Cloné el repositorio del proyecto y trabajé sobre el commit `bedab2c`. Lo primero fue ejecutar las pruebas de los cinco servicios, levantarlos y recorrer las diez guías en el orden que indica la guía general, ejecutando cada tabla de verificación y guardando la petición y la respuesta de cada paso. Después leí el código de las clases que cada guía señala para responder las preguntas de análisis.

Las verificaciones no las hice a mano en Swagger una por una: escribí un pequeño script que hace las mismas peticiones HTTP y guarda la respuesta literal. Me pareció mejor por dos razones. La primera es que así las evidencias quedan en texto, reproducibles y comparables, en lugar de en capturas de pantalla. La segunda es que varias verificaciones dependen del estado que deja la anterior —las de las extensiones encadenan datos— y con un script puedo reiniciar los servicios y repetir la secuencia completa sin equivocarme en el orden.

El entorno fue macOS sobre arm64, con Java 21.0.9 y Maven 3.9.12. Todas las evidencias están en la carpeta `evidencias/` de este mismo entregable.

1.1 Resultado de las pruebas automatizadas

```
auth-service            JwtUtilsTest 2 · AuthFlowIntegrationTest 2     Tests run: 4
estudiantes-service     EstudianteServiceImplTest 3                    Tests run: 3
cursos-service          CursoServiceImplTest 4                         Tests run: 4
matriculas-service      MatriculaControllerSecurityTest 4
                        MatriculaServiceImplTest 4                     Tests run: 8
calificaciones-service  CalificacionServiceImplTest 8                  Tests run: 8
                                                                       ──────────
                                                  27 pruebas, 0 fallos
```

Coincide con las 27 pruebas que menciona el checklist. Los cinco `BUILD SUCCESS` están en `evidencias/mvn-test.txt`.

1.2 Verificaciones ejecutadas

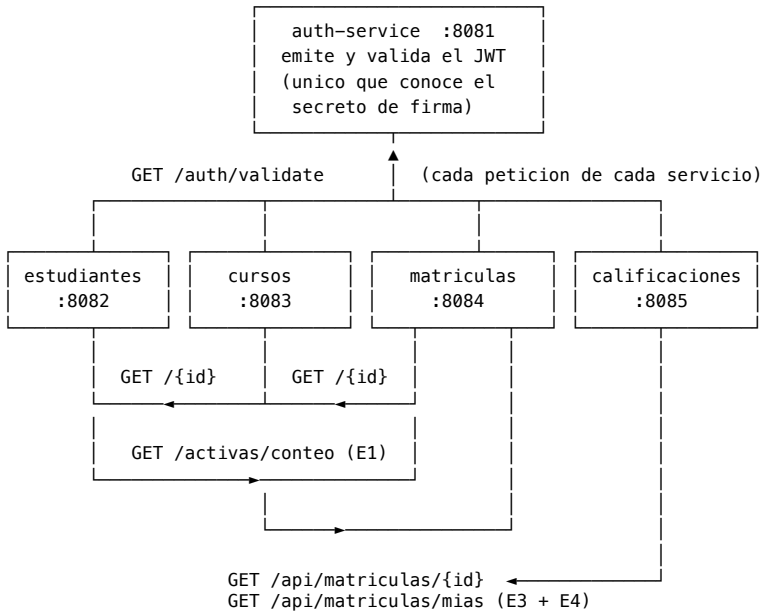
Ejecuté las tablas de las diez guías. En total son 78 pasos y todos dieron el resultado que la guía anticipaba. No encontré ninguna discrepancia entre lo documentado y lo que hace el sistema.

Guía	Pasos	Qué deja comprobado
Seguridad	7	401 sin token, 401 con token falso, 403 por rol, documentación pública
auth-service	7	login, credenciales inválidas, validación del token, token alterado
estudiantes-service	10	CRUD, correo único, validaciones, paginación, roles
cursos-service	8	código único, límite de créditos, roles, borrado
matriculas-service	10	duplicado, referencias inexistentes, anulación
E2 cupos	8	cupo lleno, liberación al anular, cupo menor que las activas
E1 integridad	9	409 al borrar con matrículas activas, conteo con filtro obligatorio

Guía	Pasos	Qué deja comprobado
E4 mis matrículas	6	`/mias` filtra por el token, 403 cruzados
E3 calificaciones	10	una nota por corte, promedio ponderado, matrícula anulada
Resiliencia	3	caída de estudiantes, de matrículas y de auth

2. La arquitectura, vista desde el código

2.1 Diagrama



Todas las flechas llevan el token del usuario (FeignAuthPropagationConfig).
Cada servicio tiene su propia base H2 en memoria y su propio Swagger.

Las flechas hacia `auth-service` son de ida y vuelta en cada petición. Las demás son llamadas Feign entre servicios de negocio: matrículas consulta estudiantes y cursos antes de registrar; estudiantes y cursos consultan a matrículas antes de borrar (eso es E1, y es lo que introduce el ciclo); calificaciones consulta matrículas para saber si puede calificar.

2.2 Las capas dentro de cada servicio

Todos los servicios repiten la misma estructura. La seguí entera con un `POST /api/estudiantes`:

```
EstudianteController.crear(@Valid EstudianteCreateDTO)      ← valida el formato de entrada
    └→ EstudianteHandler.crear(dto)                         ← convierte DTO a entidad
            └→ EstudianteServiceImpl.crear(estudiante)      ← aplica la regla de negocio
                    └→ EstudianteRepository.save(...)       ← persiste
            ←— toDto(entidad)                               ← convierte entidad a DTO de salida
    ←— ResponseBuilder.created("Estudiante creado", dto)    ← arma la respuesta uniforme
```

Lo que me parece más claro de esta separación es que cada clase tiene exactamente un motivo para cambiar. El controlador cambia si cambia el contrato HTTP; el handler, si cambia la forma del DTO; el servicio, si cambia la regla. La validación de correo único está en `EstudianteServiceImpl.crear`, línea 2 del método, y no en el controlador: por eso vale igual si mañana llega una petición desde otro sitio que no sea REST.

El `ResponseBuilder` es el que consigue que todas las respuestas tengan la misma forma `{success, message, data}`. Esto importa más de lo que parece: el cliente puede escribir un solo parser.

3. Seguridad

3.1 Evidencias

```
1  GET /api/cursos/1 sin token
   401 {"success":false,"message":"Debe enviar un token Bearer válido",
        "errorCode":"AUTH_HEADER_MISSING","status":401,"path":"/api/cursos/1"}

2  GET /api/cursos/1 con "Bearer token.inventado.abc"
   401 errorCode "TOKEN_INVALID"

3  DOCENTE  POST /api/cursos
   403 {"message":"No tiene permisos para acceder a este recurso",
        "errorCode":"AUTH_FORBIDDEN","status":403}

4  ESTUDIANTE  GET /api/cursos                      200
5  ESTUDIANTE  GET /api/estudiantes (8082)          403 AUTH_FORBIDDEN
6  GET /v3/api-docs sin token                       200
7  ADMIN  POST /api/matriculas {"estudianteId":2,"cursoId":2}
   201 — el token se propagó a estudiantes y a cursos
8  Contraseñas en la base de auth:
   $2a$10$cN7O1dx/vzscadUlT594WOsm0KI0Y6hdsOH8Vdpj4F2d3PPUixsZ6
   $2a$10$Pa8/R088ul6Rb7pH1w.5TuoZG.C.rLBJcbIRhTY618l/ZWHosRR0W
   $2a$10$FhUZgi3cv0ccg2RYIDEEkuFA7F8Um8zvNLqMBONvM8YNTDerb/1IO
9  auth-service detenido, token válido:
   401 errorCode "TOKEN_VALIDATION_ERROR"
```

Los hashes los saqué del log de Hibernate con el nivel de parámetros en TRACE, que es equivalente a mirar la tabla en la consola H2. Los tres empiezan por `$2a$10$`, miden 60 caracteres y son distintos entre sí aunque el algoritmo sea el mismo: BCrypt genera un salt aleatorio por usuario y lo guarda dentro del propio hash.

3.2 Por qué el paso 3 es 403 y no 401

Son dos momentos distintos y los decide código distinto. El 401 lo produce `AuthValidationFilter`, que corre antes que nada y solo mira el token: si no hay encabezado `Bearer`, o si `auth-service` lo rechaza, el filtro corta ahí con `reject(...)` y nunca llega al controlador.

En el paso 3 el token del docente es perfectamente válido, así que el filtro lo deja pasar y guarda en el `SecurityContext` un `UsernamePasswordAuthenticationToken` con la autoridad `DOCENTE`. Quien rechaza después es `SecurityConfig`, en esta línea:

```
.requestMatchers(HttpMethod.POST, "/api/cursos/**").hasAuthority("ADMIN")
```

El docente está autenticado pero no autorizado, y eso es exactamente lo que significa un 403. La distinción tiene consecuencias prácticas para el cliente: ante un 401 lo sensato es volver a autenticarse; ante un 403, no tiene ningún sentido, porque reintentar con el mismo usuario dará siempre lo mismo.

3.3 El costo de validar contra auth-service en cada petición

Cada petición a un servicio de negocio genera una segunda petición HTTP a `auth-service`. Eso duplica el número de llamadas del sistema y suma latencia de red a absolutamente todo.

Lo que se gana es control. Como solo `auth-service` conoce el secreto (`app.jwt.secret`), un token revocado deja de funcionar en el instante en que ese servicio lo decida, y el secreto no se reparte entre cinco servicios, con lo que hay un solo sitio del que podría filtrarse.

Si cada servicio validara la firma localmente, desaparecerían las llamadas extra y con ellas la dependencia: `auth-service` podría estar caído y el resto seguiría funcionando con los tokens ya emitidos. Lo comprobé en el paso 9 y es justo lo que pasa hoy: con `auth-service` abajo, un token perfectamente válido devuelve 401. Pero esa autonomía tiene un precio alto: el secreto estaría en los cinco servicios y revocar un token sería imposible hasta que expirase solo. Para una plataforma académica con tokens de una hora, me parece que la decisión actual está bien; en un sistema con más tráfico habría que medir si el salto de latencia compensa.

3.4 Qué no debería ir en el JWT

Nada que no se pueda hacer público. El payload va en Base64, no cifrado: pegando el token en cualquier decodificador se lee entero. Lo comprobé con el token de admin y se ve `{"sub":"admin","role":"ADMIN","iat":...,"exp":...` sin ningún esfuerzo.

La firma impide **modificarlo**, no **leerlo**. Por eso ahí no pueden ir documentos de identidad, correos, notas, ni por supuesto contraseñas. El proyecto se mantiene en lo mínimo: usuario, rol, fechas y, para los estudiantes, el `estudianteId`. Ese último es un identificador interno sin valor fuera del sistema, así que me parece aceptable.

3.5 Si el secreto se publicara

Cualquiera podría firmar un token con `"role":"ADMIN"` y el sistema lo aceptaría como legítimo, porque la validación solo comprueba que la firma cuadre con el secreto. No haría falta robar ninguna contraseña: sería un acceso

total y silencioso, y además indistinguible de un acceso real en los registros.

Hoy el secreto está en `application.yml`, es decir, dentro del repositorio. Lo primero sería sacarlo a una variable de entorno y dejar en el archivo solo un valor de desarrollo. En producción debería venir de un gestor de secretos, rotarse cada cierto tiempo y, si alguna vez se expusiera, cambiarse de inmediato, lo que invalida todos los tokens emitidos. Es la primera de las mejoras que propongo en la sección 13.

3.6 Por qué `doFilter` va fuera del `try`

El propio código lo deja comentado:

```
// Fuera de cualquier try: los errores de las capas siguientes no deben convertirse en un 401
filterChain.doFilter(request, response);
```

Si estuviera dentro del `try` que captura `FeignException`, cualquier error lanzado **más adelante** en la cadena quedaría atrapado por ese `catch`. Un caso concreto: `MatriculaServiceImpl.registrar` llama por Feign a estudiantes y, si ese servicio está caído, lanza una `FeignException` que el servicio traduce a `RemoteService–Exception` y el manejador global convierte en 502. Si el `doFilter` estuviera dentro del `try`, esa misma excepción la atraparía el filtro y el usuario recibiría un **401 "No fue posible validar el token"**, cuando su token estaba perfecto y lo que falló fue otro servicio. Es un diagnóstico equivocado que mandaría a cualquiera a revisar el token durante un buen rato.

4. auth-service

4.1 Evidencias

```
1  POST /auth/login admin/admin123      200 {"token":"eyJ...","username":"admin","role":"ADMIN"}
2  contraseña incorrecta                401 AUTH_INVALID_CREDENTIALS "Credenciales inválidas"
3  cuerpo {}                            400 VALIDATION_ERROR  {"password":"no debe estar vacío",
                                                               "username":"no debe estar vacío"}
4  GET /auth/validate (admin)           200 {"valid":true,"username":"admin","role":"ADMIN",
                                             "estudianteId":null}
5  GET /auth/validate (estudiante)      200 {"valid":true,"role":"ESTUDIANTE","estudianteId":1}
6  GET /auth/validate sin token         401 AUTH_INVALID_TOKEN
7  token con una letra cambiada         401 AUTH_INVALID_TOKEN
```

El paso 5 confirma lo que después explota E4: el `estudianteId` solo aparece en el token de quien tiene un estudiante asociado. En el de admin viene `null`.

4.2 El mismo mensaje para usuario inexistente y contraseña mala

Está hecho a propósito y se ve en `AuthServiceImpl.login`: los dos caminos lanzan `new InvalidCredentialsException("Credenciales inválidas")`.

Si el mensaje distinguiera, el login se convertiría en un verificador de usuarios: probando correos o nombres, un atacante sabría cuáles existen sin acertar ni una contraseña. Eso se llama enumeración de usuarios, y la lista resultante sirve para dirigir ataques de fuerza bruta o de suplantación.

Vale la pena señalar que el mensaje uniforme no basta del todo: si comprobar una contraseña con BCrypt tarda bastante más que no encontrar el usuario, el tiempo de respuesta delata igualmente cuál de los dos casos ocurrió. Es un detalle fino, pero existe.

4.3 Cómo verifica BCrypt si cada hash es distinto

Porque el salt no es un secreto aparte: va dentro del hash. En `$2a$10$cN701dx/vzscadUlT594W0...`, el `$2a$` dice la variante, el `10` el costo y los 22 caracteres siguientes son el salt; el resto es el resultado.

Entonces `matches(password, hash)` no descifra nada —BCrypt no es reversible—: lee el salt del hash guardado, vuelve a cifrar la contraseña recibida con ese mismo salt y compara los dos resultados. Si coinciden, la contraseña es la correcta.

El costo `10` tampoco es decorativo: es el número de iteraciones (2^10), y está ahí para que calcular cada hash sea deliberadamente lento. Un algoritmo rápido como SHA-256 permitiría probar millones de contraseñas por segundo.

4.4 Un token emitido antes de un cambio de rol

Sigue funcionando con el rol viejo hasta que expire. El token es autocontenido: lleva `"role"` dentro y `JwtUtils.parseToken` lo lee de ahí, sin consultar la tabla `usuarios`. Si a un usuario le quitan el rol de ADMIN, su token actual conserva ese permiso hasta una hora más (`app.jwt.expiration-ms`).

Hay varias salidas. La más simple es acortar la expiración, que reduce la ventana sin cambiar nada de la arquitectura. La más correcta es que `/auth/validate` consulte el rol actual en la base en lugar de leerlo del token —y esto es casi gratis aquí, porque los servicios ya preguntan a `auth-service` en cada petición; hoy se desaprovecha esa llamada—. Una tercera es llevar una lista de tokens revocados, pero eso añade estado a un servicio que hoy es cómodo precisamente por no tenerlo.

4.5 Por qué `/auth/validate` responde siempre lo mismo

Devuelve 401 sin distinguir si el token expiró, si la firma no cuadra o si está mal formado. Se pierde información útil para diagnosticar, y en desarrollo eso molesta.

Creo que es intencional. Decirle a quien envía un token que “la firma es inválida” frente a “el token expiró” le confirma que el formato iba bien y solo falló la firma, lo cual orienta a quien está probando a fabricar uno. La asimetría es clara: al usuario legítimo el detalle le ahorra unos minutos; al atacante le ahorra intentos.

5. estudiantes-service

5.1 Evidencias

1	`GET /api/estudiantes?page=0&size=5`	`200, 5 elementos, "totalElements": 50`
2	`GET /api/estudiantes/999`	`404 "Estudiante no encontrado: 999"`
3	`POST Ana Ruiz`	`201 id 51`
4	`mismo correo otra vez`	`400 "Ya existe un estudiante con el email: ana.ruiz@correo.edu"`
5	`email "no-es-correo" y edad 0`	`400 {"edad":"La edad mínima es 1", "email":"El email no es válido"}`
6	`PUT /2 con el correo del 3`	`400 "Ya existe otro estudiante con el email: ..."`
7	`DOCENTE: GET 200 · POST 403`	
8	`ESTUDIANTE: GET /1`	`403`
9	`GET /api/estudiantes/abc`	`400 "Solicitud inválida: revise los parámetros..."`

El paso 9 me parece el más interesante de la tabla. Un `abc` donde se espera un `Long` podría haber terminado en un 500 feo, o incluso en un 401 si el filtro lo hubiera capturado mal. El `GlobalExceptionHandler` atrapa `MethodArgumentTypeMismatchException` y lo convierte en un 400 con un mensaje legible, que es la respuesta correcta: el problema está en la petición del cliente.

5.2 El recorrido completo de un POST

Lo describí en la sección 2.2 con el detalle de cada clase. Lo que quiero añadir aquí es dónde está cada tipo de validación, porque son tres y es fácil confundirlas:

- **Formato**: `@Valid` sobre `EstudianteCreateDTO` en el controlador. Comprueba que el correo parezca un correo y que la edad sea mayor que 0. No toca la base.
- **Regla de negocio**: `repository.existsByEmail(...)` en `EstudianteServiceImpl.crear`. Necesita consultar la base, pero es una decisión del dominio.
- **Integridad**: la restricción `unique` sobre la columna. Es la última red.

5.3 Cuándo no basta la validación del servicio

Cuando dos peticiones llegan a la vez con el mismo correo. Las dos ejecutan `existsByEmail`, las dos reciben `false` porque ninguna ha guardado todavía, y las dos pasan a `save`. Es una condición de carrera clásica entre la comprobación y la escritura.

Ahí entra la restricción de la base de datos: la segunda inserción falla, salta una `DataIntegrityViolationException` y el `GlobalExceptionHandler` la convierte en 409. Las dos validaciones no son redundantes, hacen cosas distintas: la del servicio da un mensaje claro en el caso normal, y la de la base garantiza la invariante incluso en el caso raro.

5.4 Si el controlador expusiera la entidad

Se ahorrarían el handler y los tres DTO, y para un CRUD de cuatro campos la diferencia de código es notable. Pero se perderían tres cosas.

La primera es el control de lo que sale. Hoy `EstudianteDTO` no tiene por qué incluir todos los campos de la entidad; si mañana se agrega uno interno, no se filtra solo. La segunda es el control de lo que entra: con la entidad expuesta, un cliente podría mandar un `id` en el cuerpo de un POST y cambiar a qué registro apunta. La tercera es que el contrato de la API quedaría atado al modelo de la base: renombrar una columna rompería a todos los clientes.

En este proyecto hay un argumento extra. `matriculas-service` define su propio `EstudianteRemotoDTO` en lugar de importar el de estudiantes, y eso solo es posible porque lo que viaja es un DTO y no la entidad.

5.5 Dos DTO de entrada con los mismos campos

Hoy `EstudianteCreateDTO` y `EstudianteUpdateDTO` son idénticos, y unificarlos sería tentador. El motivo de tenerlos separados es que **van a dejar de serlo**. Crear y actualizar no son la misma operación: es normal que al crear se exija un campo que al actualizar no se puede cambiar, o al revés.

Con un DTO único, el día que aparezca esa diferencia hay que partirlo, y para entonces los clientes ya dependen del nombre que tenía. Con dos desde el principio, cada uno evoluciona por su lado sin tocar al otro. Es duplicación deliberada: cuesta unas líneas hoy a cambio de no tener que romper un contrato después.

5.6 La paginación

`GET /api/estudiantes?page=0&size=5` devolvió 5 elementos y `totalElements: 50`. Sin paginar, con 100.000 estudiantes el servicio tendría que traer las 100.000 filas a memoria, convertirlas todas a DTO y serializarlas en un único JSON de varios megabytes. Lo más probable es que reviente por memoria antes de responder, y si responde, tarda tanto que el cliente ya se rindió.

El detalle que me parece importante es que la paginación no es solo una comodidad del cliente: es lo que **acota**

el consumo de memoria del servidor. Con `size=20`, el servicio nunca carga más de 20 registros por petición, sin importar cuántos haya en la tabla. Eso hace que el uso de memoria sea predecible, que es lo que permite dimensionar la máquina.

6. cursos-service

6.1 Evidencias

```
1  GET /api/cursos                        200, tres cursos semilla con su cupoMaximo
2  POST ARQ-001                           201 id 4
3  repetir                                400 "Ya existe un curso con el código: ARQ-001"
4  creditos 11                            400 {"creditos":"Los créditos máximos son 10"}
5  PUT /1 con codigo BD2-001              400 "Ya existe otro curso con el código: BD2-001"
6  ESTUDIANTE: GET 200  ·  POST 403
7  DELETE del curso 4 (sin matrículas)    200 "Curso eliminado"
```

6.2 La duplicación con estudiantes-service

Comparando los dos servicios, estas clases son prácticamente iguales salvo por el nombre de la entidad: `ApiResponse`, `ApiErrorResponse`, `ResponseBuilder`, `AuthValidationFilter`, `CustomAuthenticationEntryPoint`, `CustomAccessDeniedHandler`, `CustomAuthenticationException`, `SecurityConfig`, `FeignAuthPropagationConfig`, `AuthClient`, `TokenValidationResponse`, `RemoteApiResponse`, `GlobalExceptionHandler`, `OpenApiConfig` y las tres excepciones del dominio. Son unas diecisiete clases repetidas en cada uno de los cinco servicios.

¿Es un problema? Depende de qué se duplica. Yo lo separaría en dos grupos.

Las piezas **transversales y estables** —el filtro de autenticación, el manejador global de errores, `ApiResponse`— son buenas candidatas para una librería compartida. Cambian poco y, cuando cambian, deben cambiar igual en todas partes. Hoy, si se descubre un fallo en `AuthValidationFilter`, hay que corregirlo cinco veces y existe el riesgo real de que alguna quede sin corregir.

Las piezas de **dominio** —`CursoServiceImpl`, los DTO, la entidad— no deberían compartirse aunque se parezcan. Ahí la duplicación es lo que permite que cursos evolucione sin coordinarse con estudiantes, que es la razón de ser de los microservicios.

Mi conclusión es que sí crearía una librería, pero solo con el primer grupo y asumiendo lo que cuesta: una librería compartida es una dependencia común, y cada versión nueva obliga a los cinco servicios a actualizarse. Hay que versionarla con cuidado para que un servicio pueda quedarse atrás sin romperse. Si esa disciplina no existe, la librería reintroduce por la puerta de atrás el acoplamiento que los microservicios querían evitar, y entonces prefiero la duplicación.

6.3 Por qué un estudiante puede ver cursos pero no estudiantes

Lo dicen las dos configuraciones. En cursos:

```
.requestMatchers(HttpMethod.GET, "/api/cursos/**").hasAnyAuthority("ADMIN", "DOCENTE", "ESTUDIANTE")
```

y en estudiantes el rol `ESTUDIANTE` no aparece en ninguna regla, así que cae en `anyRequest().authenticated()` y el `CustomAccessDeniedHandler` devuelve 403.

La razón de fondo es el principio de mínimo privilegio: cada rol recibe exactamente los permisos que necesita y ni uno más. Un estudiante necesita ver la oferta de cursos para saber en qué matricularse; no necesita, en cambio, los nombres, correos y edades de sus 49 compañeros. Y esos datos son personales: exponerlos sin necesidad es, además de un riesgo, un problema de protección de datos.

Es coherente con E4: el estudiante sí puede ver **sus** matrículas, pero no las de los demás. El criterio no es “qué recurso es” sino “qué parte de ese recurso le pertenece”.

6.4 Si un curso cambia de código

No pasa nada, y eso es deliberado. `matriculas-service` guarda `cursoId`, no el código: la entidad `Matricula` solo tiene `estudianteId`, `cursoId`, `fechaMatricula` y `estado`. Como el identificador es el numérico y no el código, cambiar `RED-001` por otra cosa no rompe ninguna matrícula.

Habría sido muy distinto si matrículas guardara el código. Tendría que enterarse del cambio y actualizar sus filas, lo que exige o bien un evento de “curso renombrado”, o bien aceptar que las matrículas viejas apunten a un código que ya no existe. Guardar la clave estable en lugar del dato legible es una decisión pequeña que evita toda esa familia de problemas.

6.5 Qué reglas dependen de otro servicio

Separando las de `CursoServiceImpl`:

Regla	¿Depende de matrículas?	Con matrículas caído
Código único al crear	No, es consulta local	Funciona
Créditos entre 1 y 10	No, es `@Min/@Max`	Funciona

Regla	¿Depende de matrículas?	Con matrículas caído
Cupo máximo obligatorio	No, es `@NotNull`	Funciona
Código único al actualizar	No	Funciona
El cupo no baja de las activas (E2)	Solo si el cupo **baja**	502 al bajarlo; subirlo o dejarlo igual funciona
No borrar con matrículas activas (E1)	Sí, siempre	502, y el curso no se borra

Lo comprobé deteniendo `matriculas-service`: `DELETE /api/cursos/1` devolvió 502 “No fue posible verificar las matrículas del curso (código remoto -1)”, y al volver a consultarlo el curso seguía existiendo.

El detalle fino está en E2: la consulta remota solo se hace cuando el cupo se reduce, porque subirlo nunca puede entrar en conflicto con las matrículas que ya hay. Así, la mayoría de ediciones de un curso no cuesta una llamada de red.

7. matriculas-service

7.1 Evidencias

```
1  POST {"estudianteId":1,"cursoId":1}   201, estado ACTIVA, id 1
2  repetir                               400 "Ya existe una matrícula activa para el estudiante
                                             y curso indicados"
3  estudianteId 999                      400 "El estudiante indicado no existe"
4  cursoId 999                           400 "El curso indicado no existe"
5  cuerpo {}                             400 {"cursoId":"El id del curso es obligatorio",
                                              "estudianteId":"El id del estudiante es obligatorio"}
6  PUT /1/anular                         200, estado ANULADA
7  repetir                               400 "La matrícula ya se encuentra anulada"
8  volver a matricular 1 en 1            201 id 2 — una anulada no bloquea una nueva
9  DOCENTE: GET 200  ·  POST 403
10 estudiantes-service detenido          502 "No fue posible consultar el estudiante (código remoto -1)"
```

7.2 Cuántas llamadas HTTP cuesta una matrícula

Contándolas sobre el código de `registrar`:

```
cliente ──► matriculas                          1
            ├─► auth /auth/validate             2   (el filtro, antes de entrar al controlador)
            ├─► estudiantes GET /{id}           3   ─┐ y cada una de estas dos dispara
            │   └─► auth /auth/validate         4    │ su propia validación de token
            └─► cursos GET /{id}                5   ─┘ en el servicio de destino
                └─► auth /auth/validate         6
```

Seis peticiones HTTP para registrar una matrícula, de las cuales **tres son validaciones del mismo token**. Las latencias se suman porque las llamadas son secuenciales, no paralelas.

En disponibilidad el efecto es peor que en latencia. La operación necesita que estén vivos matrículas, auth, estudiantes y cursos: cuatro servicios. Si cada uno tuviera por su cuenta un 99 % de disponibilidad, la combinación daría algo más del 96 %. Añadir servicios a una operación no suma disponibilidad, la multiplica a la baja.

7.3 Por qué un estudiante inexistente es 400 y un servicio caído es 502

La diferencia está en de quién es la culpa, y el código lo hace explícito en `obtenerRemoto`:

```
} catch (FeignException.NotFound ex) {
    throw new BusinessException("El " + recurso + " indicado no existe");      // → 400
} catch (FeignException ex) {
    throw new RemoteServiceException("No fue posible consultar el " + recurso +
            " (código remoto " + ex.status() + ")");                            // → 502
}
```

Un 404 remoto significa que el servicio respondió perfectamente y la respuesta fue “ese estudiante no existe”. El dato que mandó el cliente está mal, luego el error es suyo: 400. Cualquier otro fallo de Feign —conexión rechazada, tiempo agotado, un 500 del otro lado— significa que el sistema no pudo responder; el cliente no hizo nada mal: 502.

Para el cliente la diferencia es completamente práctica. Ante un 400 debe corregir los datos, y reintentar lo mismo dará siempre el mismo error. Ante un 502 los datos pueden estar bien y lo sensato es reintentar más tarde, idealmente con una espera creciente.

7.4 Por qué `EstudianteRemotoDTO` y no el DTO de estudiantes

Reutilizarlo obligaría a matrículas a depender de un artefacto de estudiantes, y eso convertiría dos servicios independientes en dos servicios acoplados en tiempo de compilación. Cualquier cambio en el DTO de estudiantes obligaría a recompilar y volver a desplegar matrículas, aunque el campo que cambió no le interese.

Definir su propio DTO deja a matrículas quedarse solo con lo que necesita. De hecho `EstudianteRemotoDTO` no tiene por qué tener todos los campos: Jackson ignora los que sobran. Si estudiantes agrega un campo, matrículas ni se entera; y el día que quite uno que matrículas sí usa, el fallo aparece en la integración y no en una cascada de compilaciones.

El costo es real: hay dos definiciones del mismo concepto que pueden desincronizarse en silencio. Esto se gestiona con pruebas de contrato, que el proyecto no tiene y que menciono en las mejoras.

7.5 Si el token no se propagara

`FeignAuthPropagationConfig` toma el encabezado `Authorization` de la petición entrante y lo copia en la saliente. Sin él, la llamada de matrículas a estudiantes saldría sin token.

La cadena sería esta. Matrículas llama a `GET /api/estudiantes/1` sin encabezado; el `AuthValidationFilter` de estudiantes no encuentra el `Bearer` y responde 401 `AUTH_HEADER_MISSING`; Feign lo convierte en `FeignException.Unauthorized`; y en `obtenerRemoto` lo atrapa esta rama:

```
} catch (FeignException.Unauthorized | FeignException.Forbidden ex) {
    throw new RemoteServiceException("No fue posible validar el " + recurso +
            ": el servicio remoto rechazó el token");
```

```
}
```

El usuario recibiría un **502**, no un 401. Y me parece correcto: su token estaba bien, el problema fue de comunicación interna. Si el proyecto devolviera 401 en este caso, mandaría a quien usa la API a revisar su token durante un buen rato buscando un problema que está en otro sitio.

7.6 Anular en lugar de borrar

El paso 8 lo demuestra: después de anular la matrícula 1, volver a matricular al mismo estudiante en el mismo curso funciona y crea la matrícula 2. Las dos coexisten, una ANULADA y otra ACTIVA.

Con borrado físico no quedaría rastro de que esa primera matrícula existió. Conservarla permite responder preguntas que no son raras en lo académico: cuántas veces un estudiante entró y salió de un curso, en qué fecha, si hubo un patrón de retiros en una asignatura concreta. También da trazabilidad para una reclamación.

Hay un efecto técnico que me parece igual de importante. Todas las reglas del sistema cuentan **matrículas activas**, no matrículas: `countByCursoIdAndEstado(cursoId, ACTIVA)` para el cupo, y la misma idea en E1 para el borrado. Eso solo funciona porque las anuladas siguen en la tabla, distinguibles por su estado. Si se borraran, el historial y las reglas tendrían que resolverse de formas distintas.

8. Extensión E2 — Cupos por curso

8.1 Evidencias

```
1  GET /api/cursos/3                       RED-001 con "cupoMaximo": 2
2  matricular estudiante 1 en curso 3      201 id 1
3  matricular estudiante 2                 201 id 2 — cupo lleno
4  matricular estudiante 3                 400 "El curso no tiene cupos disponibles (2/2)"
5  PUT /api/matriculas/2/anular            200 ANULADA
6  repetir el paso 4                       201 id 3 — se liberó un cupo
7  PUT /api/cursos/3 con cupoMaximo 1      400 "El cupo no puede ser menor que las 2
                                               matrícula(s) activa(s) del curso"
8  POST /api/cursos sin cupoMaximo         400 {"cupoMaximo":"El cupo máximo es obligatorio"}
```

El mensaje del paso 4 incluye el conteo “(2/2)”, que es un detalle pequeño pero útil: el usuario sabe de inmediato si el curso está al límite o si pidió un cupo que no existe.

8.2 Por qué el cupo está en cursos y el conteo en matrículas

Cada dato vive donde está su dueño. El cupo máximo es un atributo del curso: lo define quien lo crea y no cambia porque alguien se matricule. Las matrículas son de `matriculas-service`, que es el único que sabe cuántas hay y en qué estado.

Si matrículas guardara su propia copia del cupo, habría dos fuentes de verdad para el mismo dato. Cuando un administrador cambiara el cupo en cursos, la copia de matrículas quedaría obsoleta hasta que alguien la sincronizara, y mientras tanto el sistema aceptaría o rechazaría matrículas según un número equivocado. Peor aún: el error sería silencioso, porque las dos respuestas parecerían correctas.

El diseño actual evita eso al precio de una llamada de red por matrícula. Me parece el intercambio correcto: la llamada cuesta milisegundos y la inconsistencia cuesta confianza en el sistema.

8.3 Dos administradores y el último cupo

Sí, las dos matrículas pueden pasar. Esta es la secuencia:

```
admin A                                 admin B
──────────────────────────────────────────────────────────
countByCursoIdAndEstado -> 1
                                        countByCursoIdAndEstado -> 1
1 >= 2 ? no  -> sigue
                                        1 >= 2 ? no  -> sigue
save()  -> ahora hay 2
                                        save()  -> ahora hay 3
```

El problema está entre las líneas 43 y 53 de `MatriculaServiceImpl`: entre contar y guardar hay un hueco en el que otra transacción puede guardar. El `@Transactional` no lo evita, porque el nivel de aislamiento por defecto permite que la segunda transacción no vea la fila que la primera todavía no ha confirmado. El resultado es un curso con tres matrículas y cupo para dos, y nada en el sistema avisa.

Formas de evitarlo, de menos a más invasiva. La primera es bloquear la fila del curso con `SELECT ... FOR UPDATE` mientras se cuenta y se inserta, de modo que la segunda transacción espere; es sencillo pero serializa todas las matrículas de ese curso. La segunda es una restricción en la base, aunque “no más de N activas” no se expresa con un `unique` y haría falta un disparador. La tercera, y la que me parece mejor para microservicios, es invertir el flujo: que `cursos-service` sea el dueño de un contador de cupos ocupados y que matrículas le pida una reserva atómica; el curso decide, y quien pierde la carrera recibe el error. Cuesta más, pero es la única que no depende de que las dos operaciones compartan base de datos.

8.4 Por qué el cupo se valida después del duplicado

El orden en el código es: existencia, duplicado, cupo. Si el cupo fuera antes, un estudiante ya matriculado en un curso lleno que intentara matricularse otra vez recibiría “El curso no tiene cupos disponibles (2/2)”. Es un mensaje cierto pero inútil: lo llevaría a buscar un cupo que no necesita, porque ya está dentro.

Con el orden actual recibe “Ya existe una matrícula activa”, que le dice exactamente qué pasa. La regla general que saco es que conviene validar primero lo que es específico de esta petición y después lo que depende del estado global del sistema.

8.5 Por qué solo se consulta a matrículas cuando el cupo baja

```
if (curso.getCupoMaximo() < actual.getCupoMaximo()) {
    long activas = contarMatriculasActivas(id);
    ...
}
```

Subir el cupo nunca puede entrar en conflicto: si caben 20 y ahora caben 30, las que ya hay siguen cabiendo. Lo mismo si no cambia. La comprobación solo tiene sentido al reducirlo.

Lo que se ahorra es una llamada HTTP en la mayoría de ediciones —cambiar el nombre, el docente o los créditos

no toca el cupo—, y con ella se ahorra también su latencia y, sobre todo, la dependencia: si matrículas está caído, esas ediciones siguen funcionando. Solo se bloquea la operación que de verdad necesita el dato.

8.6 Cuándo podría llegar un cupo nulo

El código comprueba `curso.getCupoMaximo() != null` aunque en cursos-service el campo sea obligatorio. No es paranoia inútil.

El `@NotNull` protege la **entrada** de cursos-service, pero lo que matrículas recibe no es la entidad: es un `CursoRemotoDTO` construido a partir de un JSON. Hay al menos tres situaciones reales en las que ese campo llegaría nulo. La primera es que existan cursos creados antes de que E2 añadiera el campo, con la columna en `null` en la base. La segunda es que una versión futura de cursos deje de enviar el campo o lo renombre: Jackson pondría `null` sin quejarse. La tercera es que responda otra cosa —un proxy, una versión distinta del servicio— con una forma parecida pero incompleta.

Es el precio de la autonomía entre servicios: matrículas no puede dar por supuesto el contrato de cursos, solo confiar y verificar. Sin esa comprobación, un `null` daría una `NullPointerException` y un 500, en lugar de dejar pasar la matrícula sin control de cupo, que es lo que hace hoy. Dicho sea de paso, esa decisión —ante un cupo desconocido, permitir— es discutible, y en la sección 13 propongo revisarla.

9. Extensión E1 — Integridad entre servicios

9.1 Evidencias

```
1  GET /api/matriculas/activas/conteo?cursoId=3    200 {"data": 2}
2  sin filtro                                      400 "Envíe exactamente uno de los parámetros:
                                                       estudianteId o cursoId"
3  DELETE /api/cursos/3                            409 "No se puede eliminar: el curso tiene
                                                       2 matrícula(s) activa(s)"
4  DELETE /api/estudiantes/1                       409 "...el estudiante tiene 1 matrícula(s)
                                                       activa(s)"
5  DELETE /api/estudiantes/50 (sin matrículas)     200 "Estudiante eliminado"
6  matricular 5 en curso 2, DELETE /api/cursos/2   409 "...tiene 1 matrícula(s) activa(s)"
7  anular esa matrícula, DELETE otra vez           200 "Curso eliminado"
8  matriculas-service detenido, DELETE /cursos/1   502 "No fue posible verificar las matrículas
                                                       del curso (código remoto -1)"
   GET /api/cursos/1                               200 — el curso sigue existiendo
```

9.2 Las dependencias y el ciclo

Con E1 el grafo queda así:

```
estudiantes ──────► matriculas      (¿puedo borrar este estudiante?)
cursos ───────────► matriculas      (¿puedo borrar este curso?)
matriculas ───────► estudiantes     (¿existe este estudiante?)
matriculas ───────► cursos          (¿existe este curso? ¿cuál es su cupo?)
calificaciones ──► matriculas       (¿está activa esta matrícula?)
todos ────────────► auth            (¿vale este token?)
```

Hay **dos ciclos**: estudiantes <-> matrículas y cursos <-> matrículas.

Los riesgos son concretos. Al desplegar, no existe un orden correcto de arranque: cualquiera de los dos puede quedar temporalmente sin su pareja. El `depends_on` del compose no ayuda, porque solo controla el orden de inicio, no la disponibilidad. Al cambiar un contrato, el cambio tiene que coordinarse en las dos direcciones, y eso es exactamente la coordinación que los microservicios querían evitar. Y en operación, un fallo en matrículas se propaga a las eliminaciones de estudiantes y cursos, con lo que el supuesto aislamiento entre servicios desaparece.

También dificulta las pruebas: para probar la eliminación en cursos hace falta matrículas, y para probar el registro en matrículas hace falta cursos.

9.3 Eliminación lógica como alternativa

La otra opción sería no borrar: marcar el curso como inactivo y que matrículas rechace matricular en cursos inactivos.

	Verificación remota (actual)	Eliminación lógica
Acoplamiento	Alto: cursos depende de matrículas	Bajo: cada uno decide solo
Ciclo de dependencias	Lo crea	No lo crea
Disponibilidad	Borrar falla si matrículas está caído	Marcar inactivo siempre funciona
Historial	El curso desaparece; las matrículas quedan apuntando a un id inexistente	Se conserva todo
Consistencia	Inmediata	Eventual: las matrículas existentes siguen vivas
Complejidad	Una llamada remota y traducir su error	Un campo más y filtrarlo en todas las consultas

El cambio de fondo es que la eliminación lógica **invierte quién pregunta**: en vez de que cursos pregunte a matrículas antes de borrar, es matrículas quien consulta el estado del curso cuando le toca decidir. Y matrículas ya consulta cursos de todos modos, así que no añade ninguna llamada nueva ni ningún ciclo.

Me parece la mejor opción para este sistema, con un matiz: hay que decidir qué pasa con las matrículas activas de un curso que se desactiva. Lo razonable es dejarlas terminar y solo impedir las nuevas, que es como funciona en la práctica una asignatura que deja de ofrecerse.

9.4 Fail-closed frente a fail-open

Lo comprobé en el paso 8: con matrículas caído, el borrado se rechaza con 502 y el curso sigue existiendo. El sistema prefiere no hacer nada antes que hacer algo potencialmente dañino.

La opción contraria —borrar cuando no se puede verificar— dejaría matrículas activas apuntando a un curso que ya no existe. Y lo peor es que el daño es **irreversible**: el 502 se puede reintentar, un borrado erróneo no se deshace.

Fail-open sería aceptable cuando el costo de equivocarse es bajo y el de bloquear es alto. Por ejemplo, un servicio de recomendaciones: si no responde, mostrar la página sin recomendaciones es mejor que no mostrarla. Aquí no se cumple: borrar datos relacionados sin verificar corrompe el sistema, y no poder borrar un curso durante unos minutos no le hace daño a nadie.

9.5 La carrera entre el conteo y el borrado

Sí, puede pasar. Entre que cursos recibe "0 matrículas activas" y ejecuta `repository.delete(curso)`, otra transacción puede registrar una matrícula en ese curso. Es la misma clase de problema que en E2, pero repartido entre dos servicios, lo que lo hace peor: no hay transacción ni base de datos común que puedan protegerlo.

La consecuencia es una matrícula activa apuntando a un `cursoId` que ya no existe. Y como no hay llaves foráneas entre servicios, nada lo impide ni lo detecta: la fila queda ahí, y cualquier consulta que intente resolver ese curso recibirá un 404.

Con la ventana de tiempo de este sistema es poco probable, pero "poco probable" no es "imposible", y en producción las cosas poco probables ocurren. Una detección razonable sería un proceso periódico que busque matrículas activas cuyo curso o estudiante ya no exista. La eliminación lógica de la sección 9.3 elimina el problema de raíz, porque nunca desaparece nada.

9.6 Por qué 409 y no 400

Un 400 dice "tu petición está mal formada o tiene datos inválidos". Un 409 dice "tu petición es correcta, pero choca con el estado actual del recurso".

Aquí la petición es impecable: `DELETE /api/cursos/3` está bien escrita y el curso existe. Lo que impide la operación no es la petición sino el estado del sistema, que tiene dos matrículas activas. Por eso 409.

Para el cliente la diferencia cambia qué hacer. Ante un 400 hay que corregir la petición, y repetirla igual siempre fallará. Ante un 409 la petición ya es correcta: lo que hay que cambiar es el estado —anular las matrículas— y entonces la misma petición funcionará. El paso 7 de la verificación lo demuestra: anulé la matrícula y el mismo `DELETE` que daba 409 devolvió 200.

10. Extensión E4 — Mis matrículas

10.1 Evidencias

```
1  GET /auth/validate con el token del estudiante   {"role":"ESTUDIANTE","estudianteId":1}
3  ESTUDIANTE: GET /api/matriculas/mias             200, solo la matrícula del estudiante 1
4  ESTUDIANTE: GET /api/matriculas                  403 AUTH_FORBIDDEN
5  ESTUDIANTE: GET /api/matriculas/2 (de otro)      403
6  ADMIN: GET /api/matriculas/mias                  403
```

El paso 3 devolvió exactamente un elemento, el del estudiante 1, aunque en la base había matrículas de los estudiantes 1, 2 y 3. El filtrado no lo pidió el cliente: salió del token.

10.2 El diseño alternativo y por qué es vulnerable

La alternativa sería `GET /api/matriculas?estudianteId=1` abierta al rol ESTUDIANTE. Es más simple y parece equivalente, pero tiene un agujero: el `estudianteId` lo pone el cliente. Nada impide que el estudiante 1 pida `?estudianteId=2` y vea las matrículas de otro.

Eso es un IDOR, *Insecure Direct Object Reference*: el sistema usa un identificador que el usuario controla para decidir a qué datos accede, sin comprobar que le pertenezcan. Es especialmente peligroso porque los ids son secuenciales y se pueden recorrer uno a uno; con un bucle de cincuenta peticiones se tendría toda la tabla.

El diseño de `/mias` lo evita por construcción. El `estudianteId` no es un parámetro, viene del token:

```
if (usuario == null || usuario.estudianteId() == null) {
    throw new AccessDeniedException("El usuario no tiene un estudiante asociado");
}
return ResponseBuilder.ok("OK", handler.listarPorEstudiante(usuario.estudianteId()));
```

No hay forma de pedir las matrículas de otro, porque no hay dónde escribir ese “otro”. La regla que saco: lo que identifica **al usuario** debe venir del token, no de la petición.

10.3 Si se invirtiera el orden de las dos reglas

El propio código avisa:

```
// E4: debe ir ANTES de la regla general; gana la primera regla que coincide
.requestMatchers(HttpMethod.GET, "/api/matriculas/mias").hasAuthority("ESTUDIANTE")
.requestMatchers(HttpMethod.GET, "/api/matriculas/**").hasAnyAuthority("ADMIN", "DOCENTE")
```

Spring Security evalúa en orden y aplica la primera regla que coincide. Si se invirtieran, `/api/matriculas/mias` coincidiría primero con el patrón `/api/matriculas/**`, que solo permite ADMIN y DOCENTE. El estudiante recibiría 403 en el único endpoint pensado para él, y la regla específica no llegaría a evaluarse nunca: sería código muerto que parece activo.

Es un error difícil de ver leyendo, porque las dos líneas están bien escritas por separado; lo que está mal es el orden. Y se detecta solo si alguien prueba ese endpoint con ese rol, que es justo lo que hace `MatriculaControllerSecurityTest`.

10.4 Si cambia la asociación usuario–estudiante

Matrículas no se entera hasta que el estudiante vuelve a iniciar sesión. El `estudianteId` se mete en el token cuando se emite, en `JwtUtils.generateToken`, y de ahí lo lee el filtro. Mientras el token viva, lleva el valor de cuando se firmó.

Consultar la asociación en cada petición daría el valor siempre actual, a costa de una consulta más. Pero aquí ese costo es cero: los servicios ya llaman a `/auth/validate` en cada petición, y ese endpoint podría leer el `estudianteId` de la base en lugar de sacarlo del token. Hoy se desaprovecha una llamada que ya se está pagando.

Dicho esto, la asociación entre un usuario y su estudiante no cambia casi nunca —no es como el rol, que sí puede cambiar— así que en la práctica el riesgo es menor. Es una decisión defendible, pero conviene saber que existe.

10.5 `UsuarioAutenticado` como record que implementa `Principal`

```
public record UsuarioAutenticado(String username, String role, Long estudianteId) implements Principal {
    @Override public String getName() { return username; }
}
```

Implementar `Principal` es lo que permite que Spring lo inyecte con `@AuthenticationPrincipal`. Spring guarda el principal en el `SecurityContext` y esa anotación lo recupera; si fuera un objeto cualquiera, el `UsuarioAutenticado usuario` del controlador llegaría `null` y la comprobación `usuario == null` lanzaría `AccessDeniedException`. Es decir, `/mias` devolvería 403 siempre, incluso con un token correcto.

Ser un `record` aporta dos cosas. Es inmutable, lo que importa en un objeto que viaja por el `SecurityContext` y podría consultarse desde varios sitios: nadie puede cambiar el `estudianteId` después de que el filtro lo fijó. Y trae `equals` y `hashCode` por valor, lo que hace que comparar dos principals funcione como uno espera.

10.6 ¿Puede un estudiante fabricar un token con otro `estudianteId`?

No, y lo impide la firma. El token se firma con HS256 y el secreto de `app.jwt.secret`, que solo conoce `auth-service`. Cambiar un solo carácter del payload invalida la firma, y `parseClaimsJws` lanza excepción. Lo verifiqué en el paso 7 de la guía de auth: cambiando una letra del token, la respuesta pasó a 401.

Para falsificarlo habría que conocer el secreto. Por eso la sección 3.5 es tan relevante: el secreto no es un detalle de configuración, es lo único que sostiene toda la autorización del sistema. Con él, cualquiera se fabrica un token de administrador.

11. Extensión E3 — calificaciones-service

11.1 Evidencias

```
1  DOCENTE  POST {"matriculaId":1,"corte":1,"nota":4.0}   201 "registradoPor":"docente"
2  DOCENTE  POST corte 2, nota 3.0                        201
3  DOCENTE  repetir el corte 1                            400 "Ya existe una nota para el corte 1
                                                               de esta matrícula; use PUT..."
4  DOCENTE  corte 4, nota 5.5                             400 {"nota":"La nota máxima es 5.0",
                                                               "corte":"El corte debe ser 1, 2 o 3"}
5  DOCENTE  POST sobre la matrícula 2 (anulada)           400 "La matrícula 2 no está activa"
6  ADMIN    PUT /api/calificaciones/1 {"nota":4.5}        200 "registradoPor":"admin"
7  DOCENTE  GET /api/calificaciones/matricula/1           "promedioPonderado":3.75,"completa":false
8  DOCENTE  GET /api/calificaciones/matricula/99          404 "La matrícula 99 no existe"
9  ESTUDIANTE POST                                        403 AUTH_FORBIDDEN
10 ESTUDIANTE GET /api/calificaciones/mias                200, solo el resumen de sus matrículas
```

El 3.75 del paso 7 merece una comprobación. Los pesos son `{1: 0.30, 2: 0.30, 3: 0.40}` y el cálculo divide por la suma de los pesos **presentes**, no por 1.0:

`(4.5 × 0.30 + 3.0 × 0.30) / (0.30 + 0.30) = 2.25 / 0.60 = 3.75`

Sin esa normalización el resultado habría sido 2.25, que no significaría nada: parecería que el estudiante va perdiendo cuando sus dos notas son 4.5 y 3.0. Por eso el resumen trae además `completa: false`, para que quede claro que todavía falta un corte.

11.2 Qué protege cada capa de la regla "una nota por corte"

La comprobación del servicio, `existsByMatriculaIdAndCorte`, protege el caso normal y da el mensaje útil: dice qué pasó y qué hacer en su lugar, "use PUT para corregirla". La base de datos no puede dar ese mensaje.

La restricción única sobre `(matriculaId, corte)` protege el caso que el servicio no puede: dos peticiones simultáneas que pasen las dos la comprobación antes de que ninguna guarde. Es la misma carrera que en estudiantes con el correo, y la misma solución.

Resumiendo: el servicio protege la experiencia de uso, la base protege la invariante. Quitar la del servicio dejaría al usuario recibiendo un 409 genérico de integridad; quitar la de la base dejaría la puerta abierta a notas duplicadas bajo concurrencia.

11.3 Las notas de una matrícula anulada

Hoy se conservan: `verificarMatriculaActiva` solo se llama al registrar y al actualizar, no al consultar, así que `GET /api/calificaciones/matricula/{id}` sigue devolviendo las notas de una matrícula anulada.

Me parece correcto. Las notas son un hecho histórico: el estudiante las obtuvo, y la anulación posterior de la matrícula no las deshace. Borrarlas impediría responder a una reclamación o reconstruir qué pasó. Además, una matrícula puede anularse por un error administrativo que luego se corrige, y en ese caso perder las notas sería un daño innecesario.

Lo que sí haría es distinguirlas en la consulta. Hoy el resumen de una matrícula anulada se ve igual que el de una activa, y quien lo lea podría creer que esas notas cuentan para algo. Bastaría con incluir el estado de la matrícula en el resumen.

11.4 Por qué `registradoPor` sale del token

Porque es el único sitio donde el dato no se puede falsificar. Si viniera en el cuerpo de la petición, cualquiera con permiso para registrar notas podría escribir el nombre de otro docente, y el campo dejaría de servir para lo único que sirve: saber quién puso cada nota.

El token no se puede alterar sin la firma, así que el nombre que lleva es el de quien realmente se autenticó. El paso 6 lo muestra con claridad: el mismo registro pasó de `"registradoPor": "docente"` a `"registradoPor": "admin"` solo porque quien hizo el PUT fue otro usuario, sin que nadie escribiera ese nombre en ninguna parte.

La regla general es la misma de E4: **los datos sobre quién actúa vienen del token; los datos sobre qué se hace vienen de la petición.**

11.5 El 400 del paso 5 frente al 404 del paso 8

Son dos tipos de error distintos aunque los dos hablen de una matrícula.

En el paso 5 se intenta **crear** una calificación y el `matriculaId` va en el cuerpo. La matrícula 2 existe, pero está anulada: el dato que envió el cliente no sirve para esta operación. Es un error de datos de entrada, luego 400. El código lo deja explícito en un comentario: en el cuerpo de un POST, una referencia inexistente también es 400 y no 404, porque forma parte de lo que el cliente envió.

En el paso 8 se **consulta** el recurso `/api/calificaciones/matricula/99`, y ese recurso no existe. La petición está bien formada y la operación es válida; simplemente no hay nada que devolver. Eso es un 404.

La regla que saco: el 404 habla de la **ruta**, de lo que se está pidiendo; el 400 habla del **contenido** que se envió.

11.6 Por qué calificaciones no puede consultar matrículas con el token del estudiante

Porque el estudiante no tiene permiso sobre ese endpoint. `GET /api/matriculas/{id}` solo lo permiten ADMIN y DOCENTE, y el paso 5 de E4 lo confirma: con el token del estudiante devuelve 403. Como el token se propaga tal cual, calificaciones heredaría ese 403.

La solución del proyecto es elegante: en lugar de preguntar por una matrícula concreta, calificaciones llama a `GET /api/matriculas/mias` con el token del estudiante, y deja que matrículas decida cuáles son suyas. La autorización se resuelve una sola vez, en el servicio dueño del dato.

Las alternativas que se me ocurren, comparadas:

Opción	Cómo funciona	Qué cuesta
Propagar el token y usar `/mias` (actual)	Matrículas filtra por el token	Depende de que exista un endpoint "mío" en cada servicio
Token propio del servicio	Calificaciones se autentica como servicio, con permisos amplios	Hay que gestionar y rotar esa credencial, y si se filtra expone todo
Pasar el `estudianteId` por parámetro	Calificaciones pide las matrículas de un id	Reintroduce el IDOR de la sección 10.2

La primera me parece la mejor: no inventa credenciales nuevas y mantiene la decisión de autorización donde están los datos. Su límite es que obliga a diseñar un endpoint "mío" por recurso, que es justo lo que hace E4.

11.7 Qué ganó y qué costó separar el servicio

Lo que ganó. Las notas se despliegan y escalan por separado, lo que importa porque su carga es estacional: en semana de cortes se calificará muchísimo y se matriculará poco. Los permisos quedan más claros, porque el DOCENTE escribe aquí y no en matrículas. Y el modelo de matrículas no se contamina con conceptos —cortes, pesos, promedios— que no son suyos.

Lo que costó. Una base más, un despliegue más y un Swagger más. Una dependencia nueva, calificaciones -> matrículas, con su posibilidad de 502. Y la duplicación de las diecisiete clases de infraestructura que mencioné en la sección 6.2.

Mi lectura es que en un sistema real la separación se justifica por la carga estacional y por los permisos. En este proyecto concreto, con cinco servicios en una sola máquina, el costo es mayor que el beneficio inmediato; pero el ejercicio enseña exactamente lo que debe enseñar, que es cómo se paga la autonomía.

12. Docker y pruebas

12.1 Lo que pude verificar y lo que no

Las 27 pruebas automatizadas corrieron sin levantar ningún servicio y pasaron en los cinco módulos. La construcción de las cinco imágenes con `docker compose build` también se completó.

El `docker compose up` no pude terminarlo. Durante la construcción se llenó el disco de la máquina, y eso dejó el almacén de imágenes de Docker en un estado inconsistente: al volver a intentarlo, el daemon responde que no encuentra un *blob* que debería estar en `/var/lib/containerd/.../blobs/sha256/...`. Es un problema del entorno, no del proyecto: los `Dockerfile` y el `docker-compose.yml` están bien, y de hecho la construcción de las imágenes había funcionado. Repararlo exige limpiar el almacén de Docker, que afecta a otros contenedores ajenos a esta materia y por eso no lo hice por mi cuenta.

Las preguntas de análisis de esta guía sí las puedo responder desde el código, y las respondo abajo. La captura de `docker compose ps` con los cinco servicios queda pendiente.

12.2 Por qué `http://auth-service:8081` y no `localhost`

Dentro de un contenedor, `localhost` es **ese mismo contenedor**. Si `matriculas-service` buscara a auth en `http://localhost:8081`, buscaría un auth dentro de sí mismo, no lo encontraría y la conexión se rechazaría.

Compose crea una red para el proyecto con un DNS interno que resuelve el nombre de cada servicio a su IP. Por eso el compose inyecta las URLs por variable de entorno:

```
environment:
  AUTH_SERVICE_URL: http://auth-service:8081
  ESTUDIANTES_SERVICE_URL: http://estudiantes-service:8082
  CURSOS_SERVICE_URL: http://cursos-service:8083
```

Un detalle que vale la pena notar: el puerto es el **interno**, el 8081 del contenedor, no uno publicado. La publicación con `ports:` es solo para llegar desde el equipo anfitrión; entre contenedores no hace falta publicar nada. De hecho, en un despliegue serio ninguno de estos servicios debería publicar su puerto salvo el que es punto de entrada.

Que las URLs vengan por variable de entorno es lo que permite que el mismo jar funcione en local y en Docker sin recompilar: en local toman su valor por defecto con `localhost`, y en Docker lo sobrescribe el compose.

12.3 `depends_on` no espera a que el servicio esté listo

Solo controla el **orden de arranque**: inicia `auth-service` antes que los demás, pero da por iniciado un contenedor en cuanto el proceso arranca, no cuando la aplicación está lista para responder. Una aplicación Spring Boot tarda varios segundos más en levantar Tomcat y la base.

El problema concreto: `matriculas-service` puede quedar listo antes que `estudiantes-service` y, si llega una petición en esos segundos, la llamada Feign fallará y el usuario recibirá un 502 aunque todo esté bien configurado. Es un fallo intermitente, de los que aparecen una de cada diez veces y cuestan mucho de diagnosticar.

La solución estándar es un `healthcheck` y `depends_on` con condición:

```
auth-service:
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8081/v3/api-docs"]
    interval: 10s
    retries: 5
matriculas-service:
  depends_on:
    auth-service:
      condition: service_healthy
```

Así Compose espera a que el servicio responda de verdad. Conviene añadir también reintentos en el cliente: en producción nadie garantiza el orden de arranque, y un servicio debe sobrevivir a que sus dependencias se reinicien.

12.4 Por qué reiniciar borra los datos

Las bases son H2 **en memoria**: viven dentro del proceso Java y mueren con él. Reiniciar el contenedor arranca un proceso nuevo, con una base nueva y vacía, y lo único que queda son los datos que el `DataSeeder` vuelve a insertar al arrancar.

Para que sobrevivieran harían falta dos cosas. Primero, una base persistente: o H2 en modo fichero, o un motor real como PostgreSQL en su propio contenedor. Y segundo, y esto es lo que más se olvida, un **volumen** que guarde ese fichero fuera del contenedor; sin volumen, un H2 en fichero se perdería igual al recrear el contenedor, solo que sobreviviría a un simple reinicio.

Para un proyecto de clase, H2 en memoria es la elección correcta: no hay nada que instalar y cada ejecución parte de un estado conocido, lo que hace las verificaciones reproducibles. Para producción no sirve.

12.5 Copiar el jar frente a compilar dentro de Docker

Los `Dockerfile` son mínimos:

```
FROM eclipse-temurin:21-jre
WORKDIR /app
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} app.jar
EXPOSE 8081
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

La ventaja de copiar el jar es la velocidad: construir cada imagen es copiar un archivo. Y la base es `21-jre`, no `21-jdk`, lo que ya es una buena decisión: en ejecución no hace falta compilador.

El inconveniente es que la construcción **depende de que alguien haya ejecutado `mvn package` antes**, y nada en el `Dockerfile` lo garantiza. Si `target/` está vacío, falla; y lo que es peor, si contiene un jar viejo, la imagen se construye sin ningún error con código obsoleto. Es un fallo silencioso, que es la peor clase.

Un multi-stage resolvería eso compilando dentro, en una etapa que luego se descarta:

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY . .
RUN mvn -B -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /src/target/*.jar app.jar
```

La imagen final pesaría lo mismo, porque la etapa de compilación se descarta entera. Se gana que el build sea autosuficiente y reproducible: basta el código fuente, y la imagen siempre corresponde a ese código. Se pierde tiempo en cada construcción, aunque la caché de capas lo mitiga bastante.

12.6 Qué detectan y qué no las 27 pruebas

Detectan errores de lógica dentro de cada servicio: que el cupo se calcule bien, que el promedio ponderado normalice por los pesos presentes, que una matrícula anulada no bloquee una nueva, que `/mias` exija el rol correcto. Son rápidas —todas juntas tardan segundos— porque sustituyen las dependencias por dobles y no levantan nada.

No pueden detectar lo que solo aparece cuando los servicios hablan entre sí. Si `estudiantes-service` cambiara el nombre de un campo de su JSON, `MatriculaServiceImplTest` seguiría en verde, porque su mock devuelve el DTO que matrículas espera; el fallo solo saldría con los dos servicios levantados. Tampoco detectan errores de configuración —una URL mal puesta en el compose, una regla de `SecurityConfig` en el orden equivocado—, ni problemas de concurrencia como los de las secciones 8.3 y 9.5, ni nada que dependa de la red.

Ese hueco es exactamente lo que cubren las verificaciones manuales de las guías, y es la razón de que el proyecto pida las dos cosas. En un proyecto real se llenaría con pruebas de contrato entre servicios y con unas pocas pruebas de extremo a extremo sobre el sistema levantado.

13. Propuesta de mejora

De todo lo que encontré, el riesgo que me parece más grave no es el más vistoso. La condición de carrera del cupo (sección 8.3) es un fallo de corrección real, y las inconsistencias de formato que menciono abajo son molestas. Pero hay uno que, si se materializa, deja sin valor a todos los demás controles del sistema.

13.1 El riesgo: el secreto de firma está en el repositorio

En `auth-service/src/main/resources/application.yml` está `app.jwt.secret` con su valor literal. Ese secreto es lo único que impide fabricar tokens: con él, cualquiera firma un `{"sub":"quien sea","role":"ADMIN"}` que los cinco servicios aceptarán como legítimo.

Lo que lo hace tan serio es que **ningún otro control del sistema ayuda**. No hace falta robar una contraseña, así que BCrypt no sirve de nada. No hace falta saltarse el filtro, porque el token es válido. No queda rastro distinguible en los registros, porque la petición es indistinguible de una real. Y como el secreto está en el historial de Git, borrarlo de la versión actual no lo elimina: sigue recuperable en cualquier commit anterior, y en un repositorio público lo tiene quien quiera.

Es, además, el riesgo más barato de corregir de todos los que encontré.

13.2 Qué haría

Sacar el secreto a una variable de entorno, con un valor de desarrollo como respaldo:

```
app:
  jwt:
    secret: ${JWT_SECRET:clave-solo-para-desarrollo-local}
    expiration-ms: ${JWT_EXPIRATION_MS:3600000}
```

El cambio es de una línea y no toca `JwtUtils`, porque Spring ya resuelve `${...}` desde el entorno. En Docker se inyecta igual que las URLs de los servicios, que ya se hace así:

```
auth-service:
  environment:
    JWT_SECRET: ${JWT_SECRET:?hay que definir JWT_SECRET}
```

Esa sintaxis con `:?` hace que Compose se niegue a arrancar si la variable no existe, lo que evita el peor de los finales: desplegar en producción con la clave de desarrollo sin que nadie se dé cuenta.

Rotar el secreto, porque el actual ya es público. Hay que cambiarlo en cuanto se saque del repositorio; rotarlo invalida todos los tokens vivos, que con una hora de expiración es un inconveniente menor.

En producción, un gestor de secretos en lugar de variables de entorno planas, porque las variables se ven con un `docker inspect` y aparecen en los volcados de proceso.

13.3 Por qué esta y no otra

La comparé con las otras dos candidatas:

	Secreto fuera del código	Carrera del cupo (8.3)	Librería compartida (6.2)
Qué pasa si no se corrige	Suplantación total e indetectable	Un curso con una matrícula de más	Un fallo corregido en 4 de 5 servicios
Probabilidad	Alta: el repositorio es público	Baja, pero real bajo carga	Media, con el tiempo
Costo de corregir	Una línea por servicio	Rediseñar la reserva de cupo	Montar y versionar una librería
Riesgo de la corrección	Ninguno	Puede serializar las matrículas	Reintroduce acoplamiento

El cupo es un fallo de corrección y habría que arreglarlo, pero su daño es acotado y reparable a mano. La librería compartida es una mejora de mantenibilidad que puede esperar y que, mal hecha, empeora las cosas. El secreto es el único cuyo daño no tiene techo ni se detecta, y es a la vez el más barato de corregir. Por eso lo pongo primero.

Si hubiera presupuesto para una segunda, iría a la reserva atómica de cupo que describí en 8.3, porque es la única de las dos carreras que un usuario puede provocar sin proponérselo.

14. Conclusiones

Lo que más me llevo de analizar este proyecto es que casi todas las decisiones de diseño que vi son **intercambios**, no aciertos o errores. Validar el token contra `auth-service` en cada petición da control a cambio de latencia y de un punto único de falla; lo comprobé tumbando ese servicio y viendo cómo un token perfectamente válido pasaba a dar 401. Que `matriculas-service` defina su propio `EstudianteRemotoDTO` evita el acoplamiento a cambio de que los dos contratos puedan desincronizarse en silencio. E1 protege la integridad entre servicios a cambio de crear dos ciclos de dependencias. Ninguna de las tres es gratis, y lo útil del ejercicio fue poder nombrar qué se paga en cada caso.

La segunda cosa es que **los problemas interesantes aparecen en los bordes**, no en el camino feliz. Las verificaciones que más me enseñaron fueron las que rompen algo a propósito: detener un servicio y ver si el sistema falla hacia el lado seguro, pedir las matrículas de otro estudiante, bajar el cupo por debajo de las matrículas que ya existen. Las dos condiciones de carrera que encontré —la del cupo y la del borrado— no se ven ejecutando la aplicación normalmente, solo leyendo el código y preguntándose qué pasa si dos peticiones llegan a la vez.

La tercera es sobre los códigos de estado. Al principio me parecían un detalle de forma, y acabé viendo que son parte del contrato: el 400 frente al 502 de matrículas le dice al cliente si debe corregir los datos o reintentar más tarde, y el 409 frente al 400 de E1 le dice si debe cambiar la petición o cambiar el estado del sistema. Elegirlos bien es lo que permite que un cliente reaccione sin tener que leer el mensaje de error.

Por último, el detalle que más me sorprendió por pequeño. El comentario de `AuthValidationFilter` que dice que `doFilter` va fuera del `try` parece trivial hasta que uno sigue la cadena: dentro del `try`, un 502 legítimo de otro servicio se convertiría en un 401 y mandaría a quien usa la API a revisar un token que estaba perfecto. Una línea de código en el sitio equivocado habría costado horas de diagnóstico, y eso resume bastante bien por qué vale la pena leer un sistema antes de tocarlo.